

ЗВІТ

з лабораторної роботи № 4

Конструювання класів: інкапсуляція та принципи SOLID

Реквізит	Значення
Дисципліна	Конструювання програмного забезпечення
Проект	LibraryDesk — каталог бібліотеки та видача книжок
Варіант	№ 2
Виконав	Гордєєв Дмитро Леонідович
Група	ІПЗ
Дата	8 жовтня 2026 року

Репозиторій: <https://github.com/kuum-oss/LibraryDesk>

1. Мета та вихідні дані

Мета роботи — спроектувати формуляр видачі як абстрактний тип даних із перевіреними інваріантами та контрольованим життєвим циклом. Також потрібно винести зберігання, ціноутворення й сповіщення за вузькі інтерфейси та перевірити виправлення порушень SOLID тестами.

Вихідні дані. Продовжено .NET 8 рішення LibraryDesk з лабораторних робіт № 1–3. Документ — Loan, позиція — LoanItem, учасник — Reader, об'єкт каталогу — Book. Припущення: одна позиція означає один примірник на Days днів; ціна дорівнює Days × DailyRate; знижка обраної політики застосовується до кожної позиції; резервування примірників не входить до цієї роботи.

2. Завдання 1. Класова модель

Сутність / роль	Незмінність	Інваріант
Loan — формуляр	Змінний агрегат	Id > 0; ReaderId > 0; DueOn ≥ дата оформлення; позиції лише у Draft
LoanItem — позиція	sealed record	Isbn не порожній; Days > 0; DailyRate ≥ 0
Reader — читач	sealed record	Id > 0; FullName не порожнє; Email містить @
Book — книга	Змінний об'єкт	Isbn і Title не порожні; RentalFee ≥ 0; AvailableCopies ≥ 0

Стани й дозволені переходи

Зі стану	До стану	Умова
Draft	Active	Є позиції; Issue()
Draft	Cancelled	До видачі; Cancel()
Active	Overdue	today > DueOn; MarkOverdue(today)
Active / Overdue	Returned	Книги повернуто; Return()
Active / Overdue	Lost	Втрату зафіксовано; MarkLost()

Returned, Cancelled і Lost — кінцеві стани. Повна модель розміщена у docs/model.md.

3. Завдання 2. Незмінні типи та інваріанти

Обидва типи — sealed record із властивостями лише для читання. Конструктор відхиляє неприпустимі значення до появи об'єкта. `LoanItem.Amount` обчислюється щоразу з `Days` та `DailyRate` і не може розійтися з ними. Читача й позицію не потрібно змінювати після створення, тому незмінність спрощує передачу між сервісами.

LoanItem — позиція

```
public sealed record LoanItem
{
    public LoanItem(string isbn, int days,
                    decimal dailyRate)
    {
        ArgumentException
            .ThrowIfNullOrWhiteSpace(isbn);
        ArgumentOutOfRangeException
            .ThrowIfNegativeOrZero(days);
        ArgumentOutOfRangeException
            .ThrowIfNegative(dailyRate);
        Isbn = isbn;
        Days = days;
        DailyRate = dailyRate;
    }
    public string Isbn { get; }
    public int Days { get; }
    public decimal DailyRate { get; }
    public decimal Amount => Days * DailyRate;
}
```

Reader — учасник

```
public sealed record Reader
{
    public Reader(int id,
                  string fullName, string email,
                  bool hasActiveMembership)
    {
        ArgumentOutOfRangeException
            .ThrowIfNegativeOrZero(id);
        ArgumentException
            .ThrowIfNullOrWhiteSpace(fullName);
        ArgumentException
            .ThrowIfNullOrWhiteSpace(email);
        if (!email.Contains('@',
                            StringComparison.Ordinal))
            throw new ArgumentException(
                "Пошта має містити символ @.",
                nameof(email));
        Id = id;
        FullName = fullName;
        Email = email;
        HasActiveMembership = hasActiveMembership;
    }
    public int Id { get; }
    public string FullName { get; }
    public string Email { get; }
    public bool HasActiveMembership { get; }
}
```

Лістинг 1. Скорочені витяги з `LibraryDesk.Core/Domain/LoanItem.cs` і `Reader.cs`; рядки перенесено для сторінки.

Для грошей використано `decimal`; дата повернення в агрегаті має тип `DateOnly`, дата оформлення — `DateTimeOffset`. Після цього кроку рішення збиралося без попереджень.

4. Завдання 3. Інкапсуляція агрегату

Loan створюється в стані Draft. Конструктор перевіряє Id, ReaderId і DueOn. Приватний список _items відкривається лише як IReadOnlyList; AsReadOnly не дозволяє привести його назад до List і додати позицію в обхід AddItem.

```
private readonly List<LoanItem> _items = new();
public IReadOnlyList<LoanItem> Items => _items.AsReadOnly();
public LoanStatus Status { get; private set; }

public void AddItem(LoanItem item)
{
    ArgumentNullException.ThrowIfNull(item);
    EnsureStatus(LoanStatus.Draft);
    _items.Add(item);
}

public void Issue()
{
    EnsureStatus(LoanStatus.Draft);
    if (_items.Count == 0)
        throw new InvalidOperationException("Порожній формуляр");
    Status = LoanStatus.Active;
}
```

Лістинг 2. Контроль колекції та переходу Draft → Active; текст винятку скорочено.

Було	Стало	Що це дає
Status { get; set; }	private set та Issue, Return, Cancel	Клієнт не встановить довільний або зворотний стан
AddItem одразу додавав у List	AddItem перевіряє Draft	Після видачі або закриття позиції не змінюються
Items повертало List як IReadOnlyList	Items повертає AsReadOnly	Не можна обійти AddItem приведенням типу
Book.AvailableCopies { get; init; }	private set, Restock і LendCopy	Кількість примірників не стане від'ємною

Операції з файлами, консоллю чи поштою в Loan відсутні. Метод Total залишено для сумісності з попередньою моделлю; ціна з обраною політикою обчислюється сервісом.

5. Завдання 4. Інтерфейси та впровадження залежностей

`ILoanRepository` містить тільки `Add`, `GetById` і `GetAll`. `InMemoryLoanRepository` зберігає документи у `Dictionary`, відхиляє повторний `Id` та повертає копію списку. `IPricingPolicy` має один метод `PriceOf`; реалізації `StandardPricingPolicy` і `DiscountPricingPolicy` не змінюють `LoanService`.

```
public interface ILoanRepository
{
    void Add(Loan loan);
    Loan? GetById(int id);
    IReadOnlyList<Loan> GetAll();
}
public interface IPricingPolicy
{
    decimal PriceOf(LoanItem item);
}
public LoanService(ILoanRepository repository,
                  IPricingPolicy pricing, INotifier notifier)
{
    _repository = repository ?? throw new ArgumentNullException(nameof(repository));
    _pricing = pricing ?? throw new ArgumentNullException(nameof(pricing));
    _notifier = notifier ?? throw new ArgumentNullException(nameof(notifier));
}
```

Лістинг 3. Вузькі контракти й конструктор `LoanService`; залежності зберігаються у `readonly` полях.

```
ILoanRepository repository = new InMemoryLoanRepository();
IPricingPolicy pricing = new DiscountPricingPolicy(0.05m);
INotifier notifier = new ConsoleLoanNotifier();
LoanService service = new(repository, pricing, notifier);
```

Лістинг 4. Композиційний корінь у `LibraryDesk.App/Program.cs`. Щоб перейти на іншу політику, достатньо змінити один рядок створення `pricing`. Запуск показав суму 475,00 для двох днів по 250 із знижкою 5%.

6. Завдання 5. Аудит п'яти порушень SOLID

Навчальний LoanManager у LegacyLab4 адаптовано з методички до бібліотечної області. Він навмисно зберігає порушення, але не викликається з робочого застосунку. Для демонстраційного коду локально приглушено лише пов'язані попередження аналізаторів.

Принцип	Доказ у коді	Наслідок зміни вимог	Засіб
SRP	Place рахує, зберігає та сповіщає; BuildCsvReport пише файл	Зміна CSV або повідомлення зачіпає клас розрахунку	Сервіс, сховище, сповіщувач, форматувальник
OCP	if / else if за readerType: regular, vip, staff	Новий тип читача змінює Place	IPricingPolicy та окремі реалізації
LSP	ArchivedLoan : LegacyLoan; AddLine кидає NotSupportedException	Archive падає під час виклику AddLine	ArchivedLoanSnapshot містить Loan
ISP	ILoanStore оголошує 7 методів; менеджер використовує SaveToFile	Інше сховище мусить реалізувати пошту й друк	ILoanRepository та INotifier
DIP	Поля _store = new FileLoanStore(); _notifier = new SmtпNotifier()	Тест ціни залежить від файлу й консолі	Залежності через конструктор

Числа аудиту. LoanManager має 5 причин для зміни: правила ціни, збереження, сповіщення, CSV і архівування. Із 7 методів ILoanStore йому справді потрібен 1 — SaveToFile.

```
LegacyLoan archived = new ArchivedLoan(loan.Id);  
foreach (LegacyLoanLine line in loan.Lines)  
    archived.AddLine(line);
```

Лістинг 5. Конкретне порушення LSP у LegacyLab4/LoanManager.cs.

7. Завдання 6. Виправлення SOLID

Принцип	Було	Стало
SRP	LoanManager.Place і BuildCsvReport	LoanService, InMemoryLoanRepository, LoanCsvReport
OCP	if / else if у Place	StandardPricingPolicy, DiscountPricingPolicy
LSP	ArchivedLoan : LegacyLoan	ArchivedLoanSnapshot із приватним Loan
ISP	ILoanStore: 7 методів	ILoanRepository: 3; INotifier: 1
DIP	new FileLoanStore, new SmtпNotifier в полях	Інтерфейси у конструкторі LoanService

```
public sealed class ArchivedLoanSnapshot
{
    private readonly Loan _loan;
    public ArchivedLoanSnapshot(Loan loan, DateOnly archivedOn)
    {
        ArgumentNullException.ThrowIfNull(loan);
        if (loan.Status is not (LoanStatus.Returned
            or LoanStatus.Cancelled or LoanStatus.Lost))
            throw new InvalidOperationException("Лише завершений формуляр");
        _loan = loan;
        ArchivedOn = archivedOn;
    }
    public int Id => _loan.Id;
    public IReadOnlyList<LoanItem> Items => _loan.Items;
}
```

Лістинг 6. Архівний перегляд не має AddItem. Вимога кінцевого стану гарантує, що вкладений Loan більше не змінюється; текст винятку скорочено.

LoanCsvReport.BuildCsv повертає рядок без File.WriteAllText. LoanService приймає ILoanRepository, IPricingPolicy та INotifier у конструкторі; конкретні реалізації створює лише Program.cs. У робочому коді жоден сервіс не створює власне файлове сховище або сповіщувач.

8. Завдання 7. Три тести інваріантів та коміти

Додано рівно три тести ЛР № 4 в окремому Lab4InvariantTests. Попередні характеристичні й регресійні тести збережено: загалом у рішенні проходить 11 тестів.

```
[Fact]
public void LoanItem_WithZeroDays_ThrowsOutOfRangeException()
{
    Assert.Throws<ArgumentOutOfRangeException>(
        () => new LoanItem("ISBN-1", 0, 100m));
}

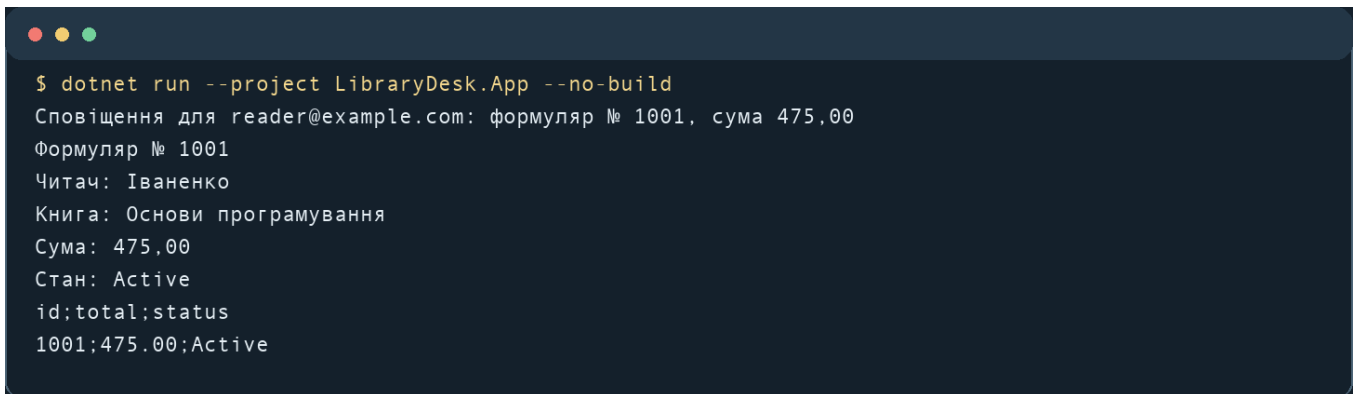
[Fact]
public void Loan_BeforeIssue_RejectsReturn()
{
    Loan loan = NewLoan(1);
    Assert.Throws<InvalidOperationException>(() => loan.Return());
}

[Fact]
public void LoanService_WithInjectedTenPercentPolicy_Returns180()
{
    ILoanRepository repository = new InMemoryLoanRepository();
    IPricingPolicy pricing = new DiscountPricingPolicy(0.10m);
    LoanService service = new(repository, pricing, new TestNotifier());
    Loan loan = NewLoan(2);
    loan.AddItem(new LoanItem("ISBN-2", 2, 100m));
    Assert.Equal(180m, service.TotalOf(loan));
}
```

Лістинг 7. Три модульні тести; NewLoan і TestNotifier — локальні допоміжні типи тесту.

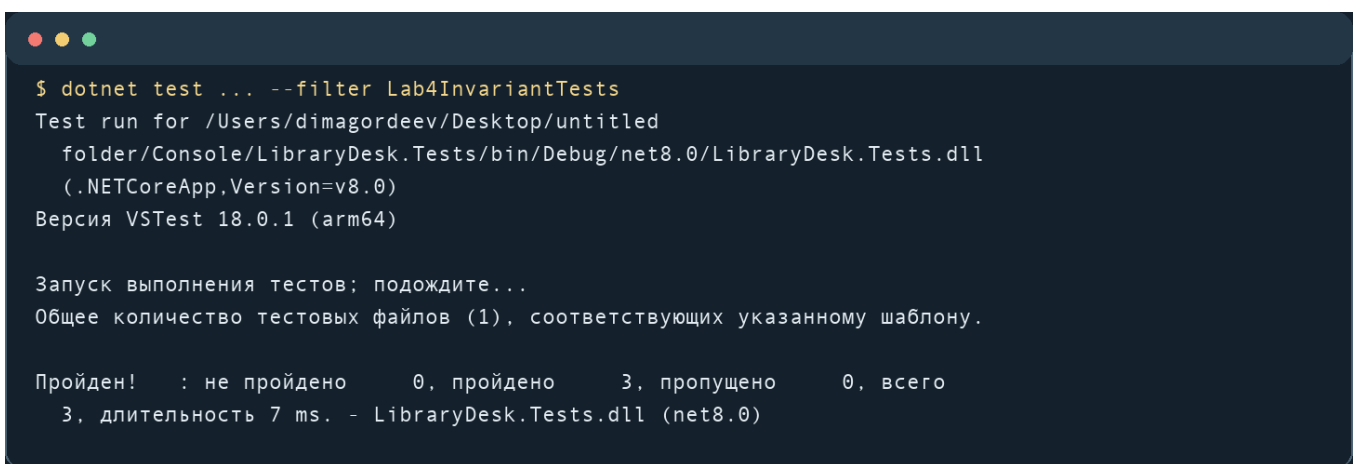
Перевірка	Результат
dotnet build LibraryDesk.sln --no-restore	Успіх; 0 попереджень; 0 помилок
dotnet test LibraryDesk.sln --no-build --no-restore	11 із 11 пройшли
Фільтр Lab4InvariantTests	3 із 3 пройшли
dotnet format --verify-no-changes	Успіх; змін не потрібно

9. Знімки виведення команд



```
$ dotnet run --project LibraryDesk.App --no-build
Сповіщення для reader@example.com: формуляр № 1001, сума 475,00
Формуляр № 1001
Читач: Іваненко
Книга: Основи програмування
Сума: 475,00
Стан: Active
id;total;status
1001;475.00;Active
```

Рисунок 1. Запуск застосунку підтверджує оформлення формуляра, розрахунок 475,00 і CSV без запису файлу.

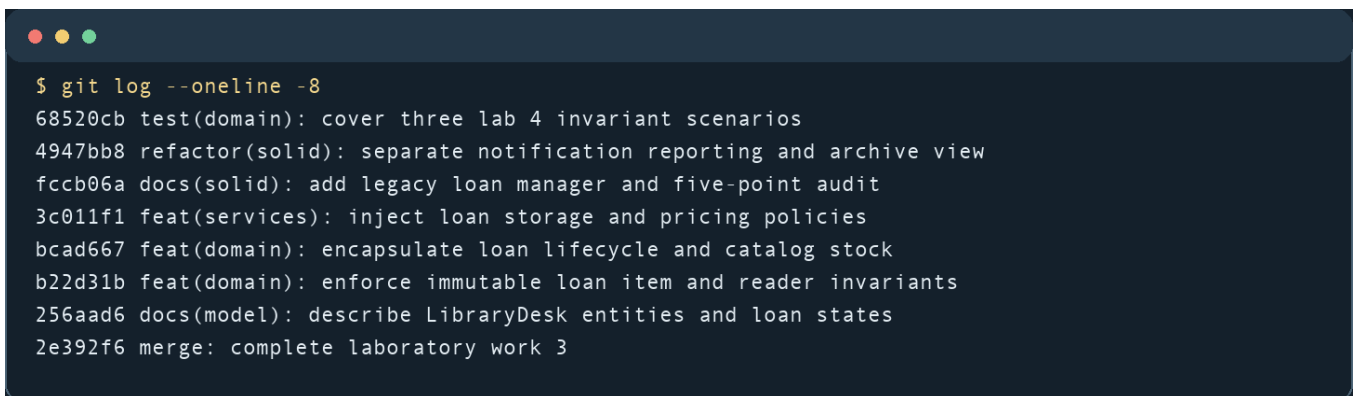


```
$ dotnet test ... --filter Lab4InvariantTests
Test run for /Users/dimagordeev/Desktop/untitled
  folder/Console/LibraryDesk.Tests/bin/Debug/net8.0/LibraryDesk.Tests.dll
  (.NETCoreApp,Version=v8.0)
Версія VSTest 18.0.1 (arm64)

Запуск выполнения тестов; подождите...
Общее количество тестовых файлов (1), соответствующих указанному шаблону.

Пройден!   : не пройдено      0, пройдено      3, пропущено      0, всего
3, длительность 7 ms. - LibraryDesk.Tests.dll (net8.0)
```

Рисунок 2. Ізольований запуск трьох тестів ЛР № 4: усі пройшли.



```
$ git log --oneline -8
68520cb test(domain): cover three lab 4 invariant scenarios
4947bb8 refactor(solid): separate notification reporting and archive view
fccb06a docs(solid): add legacy loan manager and five-point audit
3c011f1 feat(services): inject loan storage and pricing policies
bcad667 feat(domain): encapsulate loan lifecycle and catalog stock
b22d31b feat(domain): enforce immutable loan item and reader invariants
256aad6 docs(model): describe LibraryDesk entities and loan states
2e392f6 merge: complete laboratory work 3
```

Рисунок 3. Історія Git: окремі коміти для моделі, типів, агрегату, DI, аудиту, виправлень і тестів.

10. Висновки

1. У `LoanItem` і `Reader` значення тепер перевіряються в конструкторі; після створення клієнт не може зіпсувати ці об'єкти.
2. `Loan` приховує список позицій і дозволяє лише переходи з таблиці станів; повернення до `Draft` після видачі неможливе.
3. Нова політика знижки додається окремою реалізацією `IPricingPolicy`. Для вибору іншого правила у поточному застосунку змінюється один рядок композиційного кореня.
4. Сховище, сповіщення й форматування CSV відокремлено від розрахунку. Зміна формату звіту або способу зберігання більше не вимагає правити `LoanService`.
5. Архівний перегляд використовує композицію та не обіцяє операції `AddItem`, які він не підтримує. Три спеціальні тести підтвердили захист інваріантів і підстановку політики; усі 11 тестів рішення пройшли.

Репозиторій: <https://github.com/kuum-oss/LibraryDesk>

Робоча гілка: `lab04-encapsulation-solid`

Файли реалізації: `LibraryDesk.Core/Domain`, `Abstractions`, `Services`, `Pricing`, `Storage`, `Reports`; `LibraryDesk.App`; `LibraryDesk.Tests/Lab4InvariantTests.cs`.